UNIVERSITÀ DEGLI STUDI DI BRESCIA

Department of Information Engineering

Master's Degree in Computer Science and Engineering

Optimization Algorithms – Final Report

Development of a Tabu Search Metaheuristic for the Flying Sidekick Traveling Salesman Problem

Professor:
Renata Mansini

Components:
Singh Sahiljit (Matr. 740552)
Lublanis Matteo (Matr. 736418)

Academic Year 2025/2026

Contents

1 Introduction **2**

2 Tabu Search **3**
2.1 Start 3
2.2 Moves 3
2.3 Aspiration Criterion 3
2.4 Long and Short Term Memory 4
2.5 Tabu Tenure 4
2.6 The 4 Pillars 4

3 Benchmark Instances Evaluation **7**

4 Conclusions and Future Works **9**
4.1 Future Extensions 9

1 Introduction

The Flying Sidekick Traveling Salesperson Problem (FSTSP) is a routing problem that models a mixed truck-drone delivery system to serve customers. In the FSTSP, a truck and a drone work simultaneously to serve a set of spatially distributed customers. The truck acts as a mobile depot for the drone, since while they are traveling the drone can launch from a certain node, serve a customer in another point and then return on the truck at a recovery point. This synchronization allows for simultaneous deliveries, potentially saving time. However, the problem introduces complex spatial and temporal constraints. The drone has a strictly limited flight endurance (battery life), certain customers might be restricted exclusively to truck deliveries due to parcel size or weight constraints, and the synchronization between the two vehicles requires precise timing calculations.

Because of the interdependencies between the truck's routing sequence and the drone's dispatch configurations, the FSTSP is NP-hard. Exact mathematical solvers struggle to find optimal solutions within reasonable time frames as the number of customers grows, making metaheuristics the most viable approach.

In this document, we showcase a Tabu Search metaheuristic that we developed to solve the FSTSP. Keywords of our metaheuristics are: Short and Long Term Memory, diversification, intensification, greedy algorithms such as Cheapest Insertion, dynamic Tabu tenure.

This document is divided into three main sections:

1. **Heuristic Algorithm:** the algorithm is explained in detail and a pseudo-code representation is also provided.

2. **Benchmark Instances Evaluation:** the algorithm is executed 10 times across a diverse set of standard FSTSP benchmark instances, we then show the results obtained, evaluating the objective values and the gap against the known optimal solutions obtained with Gurobi.

3. **Conclusions:** we draw final conclusions from our empirical results and we propose a few ideas to extend the project in the future.

2 Tabu Search

The Tabu Search algorithm is a metaheuristic, which means that it is a local search algorithm that uses special techniques to escape from local optima and avoid cycling during the execution. The moves allowed can also deteriorate our objective function value in order to let us explore the solution space better. By using a Tabu List, that contains all the forbidden moves, we can avoid visiting a local optimum from which the algorithm has just escaped through a worsening move. The key points of a Tabu Search algorithm, which we will discuss into details, are: **long and short term memory, tabu move, tabu tenure, aspiration criteria**.
In our algorithm, we didn't use any Critical Event Memory (CEM), so we won't talk about it.

2.1 Start

Our Tabu Search starts from an initial solution obtained in two phases. First, a greedy heuristic (either Cheapest Insertion or Nearest Neighbor) is applied to generate a truck-only solution. Then, drone sorties are added to the truck tour in a greedy manner, inserting as many feasible sorties as possible. Among the solutions generated by the two heuristics, we select the one that yields the smallest objective function value.

2.2 Moves

We implemented 3 main types of moves:

- 2-Opt: it selects two non-adjacent edges in the route and it swaps them to remove intersections;
- Drone to Truck: it changes the edges so that a customer is visited by the truck instead of the drone;
- Truck to Drone: it changes the edges so that a customer is visited by the drone instead of the truck.

During the implementation, other moves were proposed (like Swap, a move that changes the order of the customers visited by the truck, and Relocate, which changes a single node and it moves in another position of the route), but they made our algorithm computationally heavy, so we decided to discard them.

2.3 Aspiration Criterion

The aspiration criterion adopted for the short-term memory allows a tabu move if it produces a solution with a better objective value than the incumbent solution.

2.4 Long and Short Term Memory

The Short Term Memory we used for our algorithm is recency based: everytime a new solution is found, the edges and the drone sorties that were removed from the previous solution enter in a tabu state, so that they can not be added again immediately afterwards.

Our Long Term Memory takes track of:

- Transition: every time an edge or a drone sortie has been added/removed its transition counter value is incremented;
- Residency: check in how many solutions an edge or a drone sortie has been found;
- Best solutions: A list of the last 10 optimal solutions found.

The LTM is then used for two main tasks:

1. To build up a new solution from the best solutions;
2. To ruin and recreate a solution starting from information from residency and transition measures.

2.5 Tabu Tenure

Tabu Tenure is the duration j of the tabu-active state of a move.In our algorithm, Tabu Tenure is a Systematic dynamic tenure, which means that it's changed in specific contexts. Initially it is $0.2 \times$ numberOfClientToVisit We decided to set Tabu Tenure as following:

- Low tenure: when a new best solution is found, we need to intensify the research on that area, to see if we can get better solutions;
- Medium tenure: normal Tabu Tenure maintained during the research;
- High tenure: when we restart the research, we want to diversify as much as possible, so that we can explore the solution space.

2.6 The 4 Pillars

Tabu Search is structured around four fundamental memory dimensions, known as the *4 pillars*: Recency, Frequency, Quality, and Influence. Each pillar addresses a different aspect of the search history and guides the algorithm toward better solutions.

Recency
Recency is managed by the Short Term Memory, implemented as two separate Tabu Lists: one for truck edges and one for drone sorties. Every time a move is applied, the edges and sorties that were *removed* from the current solution are added to the respective tabu list for a duration equal to the current Tabu Tenure. This prevents the algorithm from immediately undoing a move and cycling back to a recently visited solution. A small random perturbation is added to the tenure at each insertion to break symmetry and avoid periodic cycling.

Frequency
Frequency is managed by the Long Term Memory through two counters: Residency e Transition. These counters are exploited during the Ruin and Recreate phase to guide the greedy reinsertion of removed nodes. For each candidate insertion, a penalty score is computed for the two edges it would introduce:

$$\texttt{penaltyScore} = \max\left(0,\ \frac{\text{residency}(e)}{t} - \frac{\text{transition}(e)}{t}\right)$$

where t is the global iteration counter, used to normalize raw frequencies into rates. An edge scores high when it has appeared frequently in past solutions but has rarely changed, meaning it has been *overexploited* by the search. Such edges receive an additional cost penalty during reinsertion, discouraging the reconstructed solution from reproducing the same structural patterns. This implements a **diversification** strategy that complements the short-term tabu restrictions with a long-term frequency-based guidance, steering the search toward unexplored regions of the solution space.

Quality
Quality is managed by maintaining an *elite pool* inside the Long Term Memory, which stores up to 10 of the best distinct solutions found during the entire search. This pool is exploited in the Elite Recombination: when the global stagnation counter exceeds its threshold, the algorithm selects up to 4 solutions from the elite pool and extracts the truck edges that appear most frequently among them, so this biases the reconstruction toward proven good structures while still allowing exploration through a controlled ruin phase.

Influence
In our project, we implemented the influence pillar by monitoring the stagnation of the search process (i.e., the number of iterations since the last improvement) and applying an appropriate move accordingly. When a high level of stagnation is detected, we apply a higher-influence move that perturbs the current solution more significantly than standard moves.

```
TabuSearch(instance, LTM, truckTabuList, droneTabuList, tabuTenure)

  solution = greedySolution(instance)
  incumbent = copy(solution)
  tabuTenure = initialTenure
  counterLocal = 0
  counterGlobal = 0
  restartCount = 0

  while (timeElapsed < 10 minutes) do
    bestMove = evaluateNeighborhood(twoOpt, truckToDrone, droneToTruck)
    solution = applyMove(bestMove)
    iterationHappened(truckTabuList, droneTabuList)
    LTM.updateFrequencies(solution)

    if (solution.obj < incumbent.obj) then
      incumbent = copy(solution)
      LTM.updateBestSolutions(incumbent)
      counterLocal = 0
      counterGlobal = 0
      restartCount = 0
      tabuTenure = LOW_TENURE # intensification
    else
      counterLocal = counterLocal + 1
      counterGlobal = counterGlobal + 1

    if (counterLocal > LOCAL_THRESHOLD) then
      solution = ruinAndRecreate(solution, ruinPercentage, LTM)
      tabuTenure = MEDIUM_TENURE

    if (counterGlobal > GLOBAL_THRESHOLD) then
      restartSearch()
      emptyTabuLists(truckTabuList, droneTabuList)
      counterGlobal = 0
      counterLocal = 0
      if (restartSearch is Elite) then
        tabuTenure = (LOW_TENURE)
      else
        tabuTenure = (HIGH_TENURE) # diversification


  done

  return incumbent
```

1: Pseudocode

3 Benchmark Instances Evaluation

In this section, we will showcase the results we got from our heuristic algorithm and we will analyze how well it works and how bad it works under certain circumstances. The computer we used to run the algorithm has the following specifications:

- AMD Ryzen 7 5700X 8-Core, 3.40GHz - 4.60GHz
- 32 GB 3200MHz RAM

In the following table, it's shown the results we got by running the instances 10 times in a row:

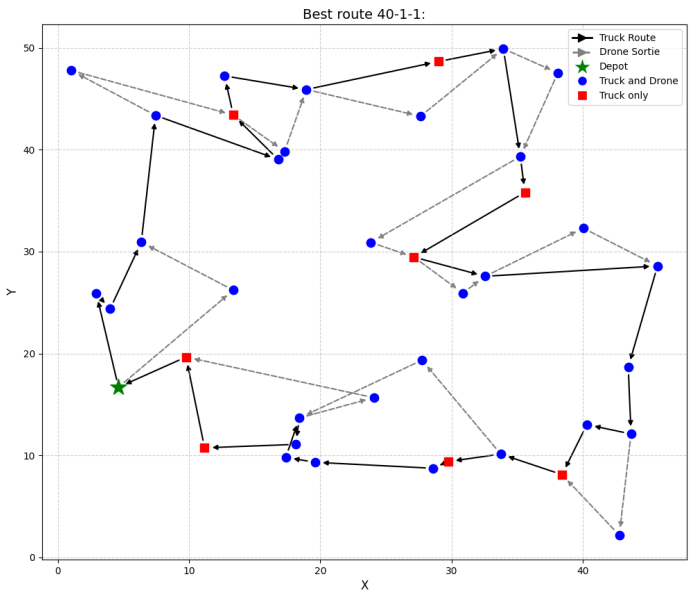
Instance	Gurobi Bd	Best Bd	Mean	Variance	Gap (%)
40-1-1	241	**241**	242.30	1.61	0.54
40-1-2	210	212	216.30	4.41	3.00
60-1-1	321	**320**	**321.0**	1.80	0.00
60-1-2	282	286	290.70	13.41	3.09
60-2-1	305	**304**	**305.00**	0.20	0.0
60-2-2	286	296	298.80	2.16	4.48
80-1-1	357	**346**	**355.80**	10.76	-0.34
80-1-2	345	**321**	**326.30**	10.01	-5.42
80-2-1	385	**380**	386.90	13.29	0.49
80-2-2	375	**370**	376.00	14.20	0.27

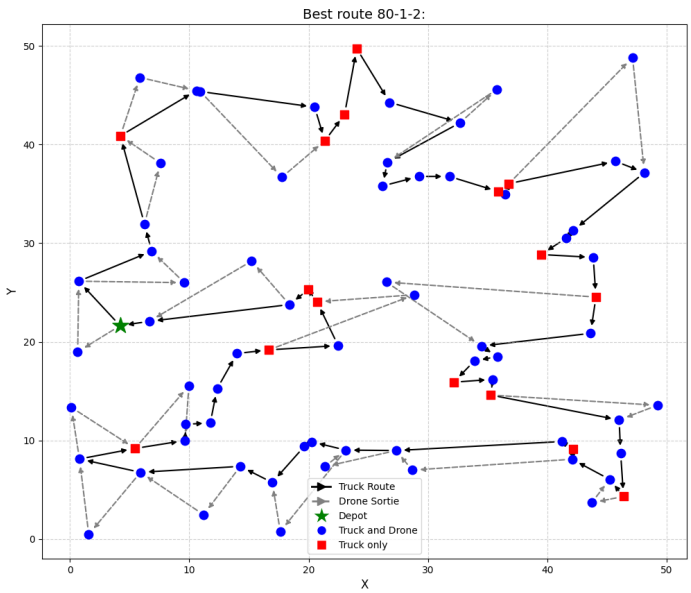
where:

- Gurobi Bd: best objective value found by Gurobi for the instance;
- Best Bd: best objective value found by the Tabu Search for the instance;
- Mean: mean objective value found for the instance.
- Variance: how much our solution differ from the mean
- Gap: mean gap obtained by the Tabu Search.

So over all instances we obtained a gap of **0,611%**.

Here are two solutions visualized into a graph:





4 Conclusions and Future Works

The computational results demonstrate the effectiveness and robustness of our Tabu Search for the FSTSP, achieving an average gap below 5% or even a negative gap across all benchmark instances. We can also see that the algorithm outperformed Gurobi on some instances such as `80-1-2` (321 against 345). While specific distribution of customers caused minor difficulties on instance `60-2-2` (4.48% gap), the overall performance confirms the framework's reliability and computational speed.

4.1 Future Extensions

The modular architecture of the algorithm opens up several possibilities for future enhancements:

- **Matheuristic Framework (Kernel Search)**.
- **Different Neighborhood Operators**.
- **Critical Event Memory (CEM)**.